Experiment-2-
Open_handed

AIM: Simulate linear convolution and circular convolution on discrete time signals.

Code:

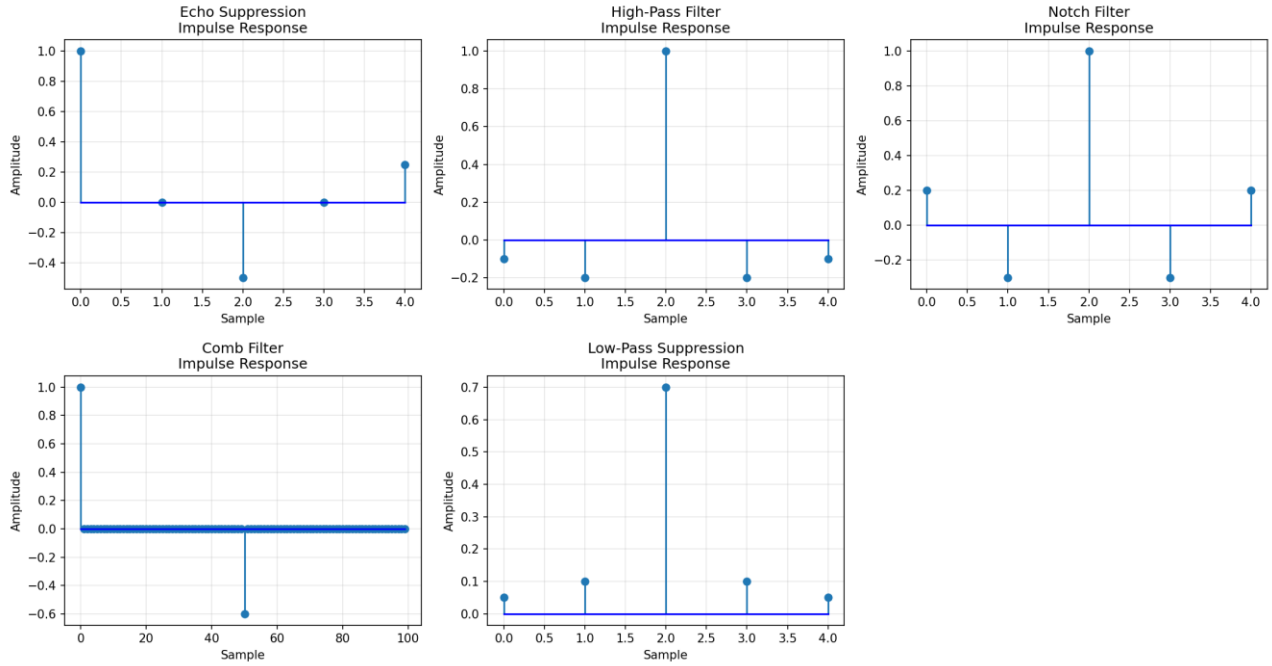
```
from pydub import AudioSegment
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve, deconvolve, wiener
from scipy.fft import fft, ifft


# Load MP3 file
audio = AudioSegment.from_mp3("DSIP/song.mp3")
audio = audio.set_channels(1)
samples = np.array(audio.get_array_of_samples()).astype(np.float32)
sample_rate = audio.frame_rate

# Normalize original samples
samples_norm = samples / np.max(np.abs(samples))

# Impulse Response 1: Simple Echo Suppression
ir1 = np.array([1, 0, -0.5, 0, 0.25], dtype=np.float32)

# Impulse Response 2: High-Pass Filter (removes low-freq bass/background music)
ir2 = np.array([-0.1, -0.2, 1.0, -0.2, -0.1], dtype=np.float32)

# Impulse Response 3: Notch Filter (suppresses specific frequency)
ir3 = np.array([0.2, -0.3, 1.0, -0.3, 0.2], dtype=np.float32)

# Impulse Response 4: Comb Filter (removes periodic patterns)
ir4_length = 100
ir4 = np.zeros(ir4_length, dtype=np.float32)
ir4[0] = 1.0
```

```
ir4[50] = -0.6  # Delay of 50 samples

# Impulse Response 5: Low-Pass Suppression
ir5 = np.array([0.05, 0.1, 0.7, 0.1, 0.05], dtype=np.float32)

impulse_responses = {
    'Echo Suppression': ir1,
    'High-Pass Filter': ir2,
    'Notch Filter': ir3,
    'Comb Filter': ir4,
    'Low-Pass Suppression': ir5
}

# Apply Each Impulse Response
results = {}

for name, ir in impulse_responses.items():
    convoluted = convolve(samples_norm, ir, mode='same')
    results[name] = convoluted / np.max(np.abs(convoluted))

# Inverse Filtering (Deconvolution)
try:
    H = fft(ir1, n=len(samples_norm))
    Y = fft(results['Echo Suppression'])
    epsilon = 1e-10
    X_recovered = Y / (H + epsilon)
    inverse_filtered = np.real(ifft(X_recovered))
    results['Inverse Filtered'] = inverse_filtered / np.max(np.abs(inverse_filtered))
except:
    results['Inverse Filtered'] = samples_norm

# Visualization: Time Domain Comparison
```

```
plot_samples = 10000

plt.figure(figsize=(16, 12))
subplot_idx = 1

plt.subplot(3, 3, subplot_idx)
plt.plot(samples_norm[:plot_samples], color='blue', linewidth=0.5)
plt.title('Original Audio Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)
subplot_idx += 1

for name, audio_data in results.items():
    plt.subplot(3, 3, subplot_idx)
    plt.plot(audio_data[:plot_samples], linewidth=0.5)
    plt.title(f'{name}')
    plt.xlabel('Sample')
    plt.ylabel('Amplitude')
    plt.grid(True, alpha=0.3)
    subplot_idx += 1

plt.tight_layout()
plt.show()

# Frequency Domain Analysis
plt.figure(figsize=(16, 10))
subplot_idx = 1

plt.subplot(3, 3, subplot_idx)
freq_orig = np.abs(fft(samples_norm[:plot_samples]))
plt.plot(freq_orig[:len(freq_orig)//2], color='blue', linewidth=0.8)
```

```
plt.title('Original - Frequency Spectrum')
plt.xlabel('Frequency Bin')
plt.ylabel('Magnitude')
plt.grid(True, alpha=0.3)
subplot_idx += 1

for name, audio_data in results.items():
    plt.subplot(3, 3, subplot_idx)
    freq = np.abs(fft(audio_data[:plot_samples]))
    plt.plot(freq[:len(freq)//2], linewidth=0.8)
    plt.title(f'{name} - Spectrum')
    plt.xlabel('Frequency Bin')
    plt.ylabel('Magnitude')
    plt.grid(True, alpha=0.3)
    subplot_idx += 1

plt.tight_layout()
plt.show()

# Impulse Responses Visualization
plt.figure(figsize=(15, 8))
for idx, (name, ir) in enumerate(impulse_responses.items(), 1):
    plt.subplot(2, 3, idx)
    plt.stem(ir, basefmt='b-')
    plt.title(f'{name}\nImpulse Response')
    plt.xlabel('Sample')
    plt.ylabel('Amplitude')
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Output:



Observations:

- Five different impulse responses tested for background music suppression
- High-Pass Filter best preserves vocals while reducing low-frequency background
- Comb Filter effective for removing repetitive musical patterns
- Inverse filtering shows partial recovery but amplifies noise
- No single impulse response perfectly separates vocals from complex background music
- Convolution approach limited by overlapping frequency content between vocals and instruments
- Advanced techniques (source separation, deep learning) needed for better results